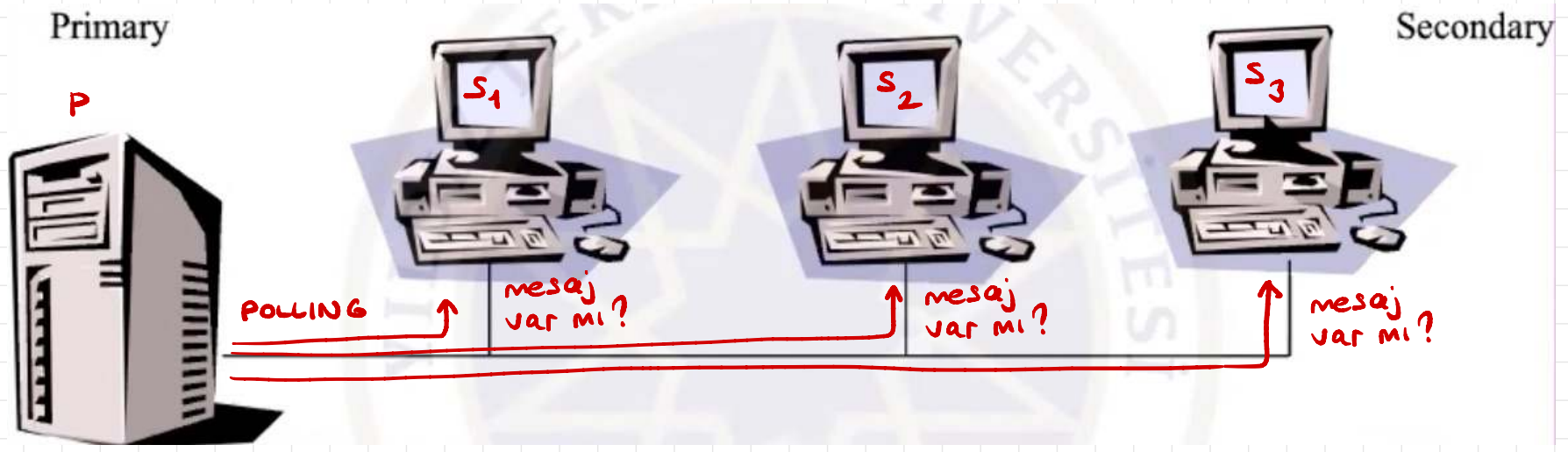
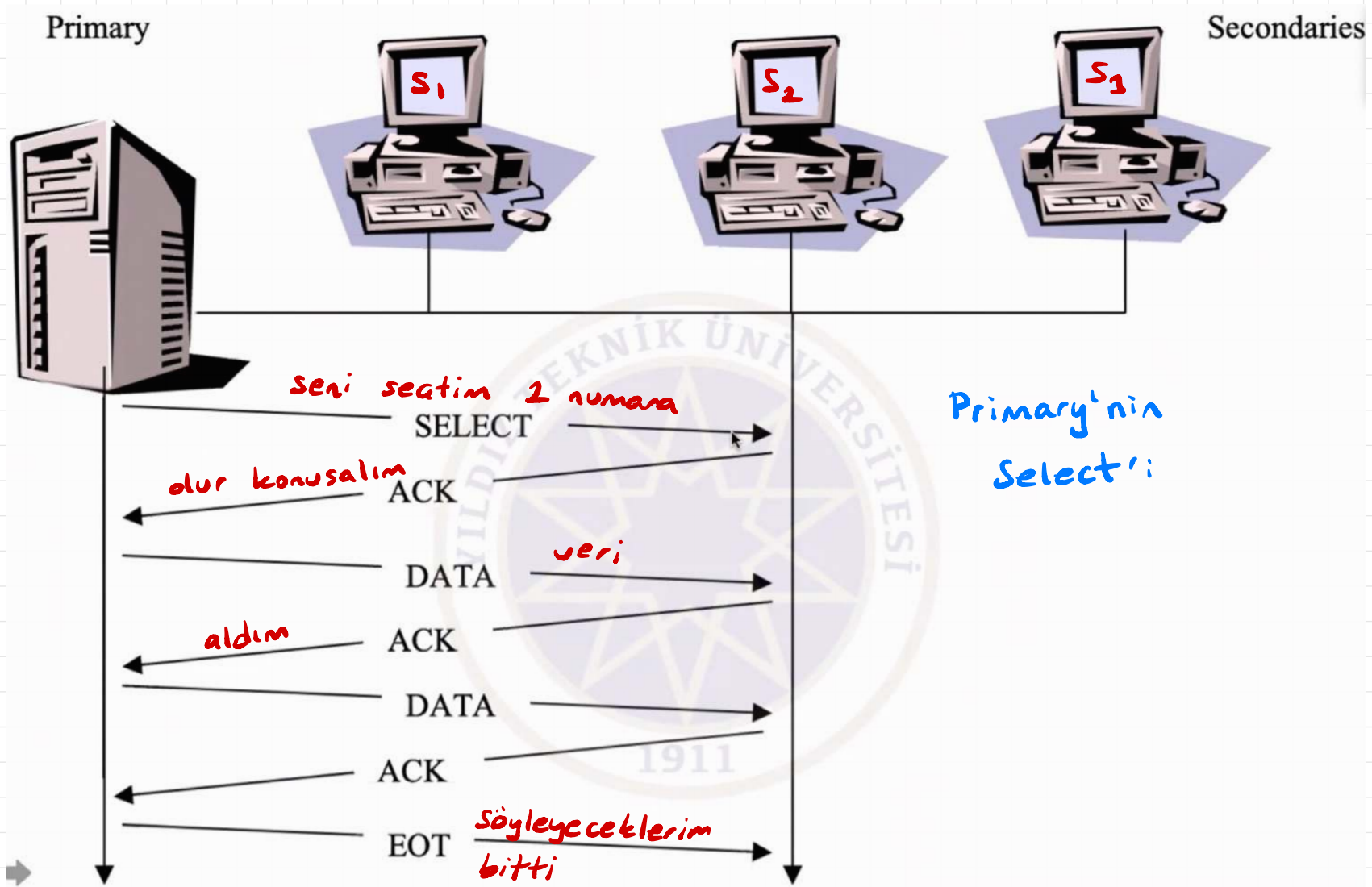




P POLLING : S1 mesajın var mı?
 S2 mesajın var mı? *var.* S1'e mesajım var.
 S3 mesajın var mı?

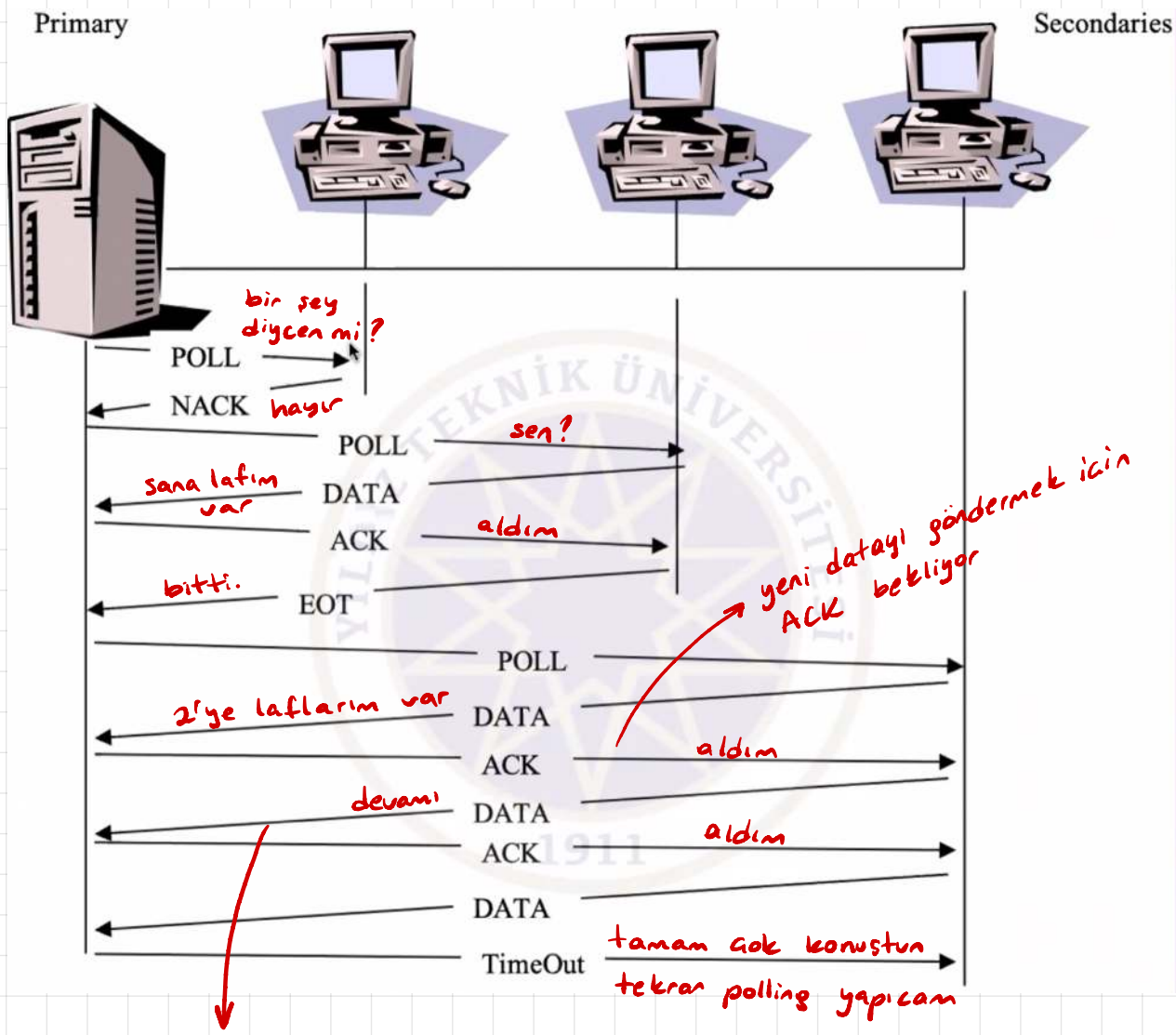
S2 ^{data} → P

P SELECT : P ^{select} → S1 P ^{S2'nin mesajları} → S1



Bir data gidip bir cevap geliyor.

Bu nedenle dönen ACK / NACK hangi data mesajına ait biliniyor.



bu data S₃ ve P arasında olmak zorunda değil.

Belki de S₃, S₂ için data gönderiyor.

Durum böyleyse P sonra SELECT yapıp S₂'ye aldığı dataları iletir.

Timeout'tan sonra tekrar sıra geldiğinde kaldığı yerden devam gelince.

Kaldığı yer demek en son ACK'yi aldığı mesajdan bir sonraki mesaj, sıradaki.

POLL karşılığında

NACK : söyleyecek bir şeyim yok

DATA : data

DATA gelince

ACK : datayı aldım OK.

NACK : datada hata var. tekrar yolla

Timeout:

Çok konuştun. Bakalım diğerlerinin söylemek istediği bir şey var mı? 100 makine varsa diyelim.

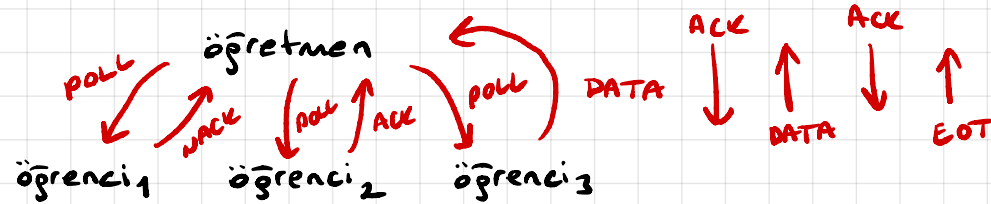
Sanıyede hepsine 10ms ayırdık diyelim. 10 ms'de lafını bitiremeyen timeout mesajını alınca susar. Bir sonraki sıra geldiğinde, kaldığı yerden datayı yollar.

Tüm makinelere eşit zaman ayrılmak zorunda değil. Bazılarının rolü daha büyükse onlara daha çok vakit ayrılıyor olabilir.

Benzetme:

primary = öğretmen

secondary = öğrenci



ACK almadıysam, o datayı alınmamış kabul ederim.

FLOW CONTROL

Overwhelm: Karşı tarafı pakete boğmak. Bufferını doldurup

Buffer: Elindeki veriyi uğrasırken, gelen veriler burada bekletilir. FIFO memory gibi.

Henüz istenememiş veriler burada bekliyor. İşlemek = hata kontrolü, düzeltmesi vs.

Zoom'da bir an hızlıca video oynaması bufferda biriken verinin boşaltılma anıdır.

İki basit teknik var:

- Stop and wait
- Sliding Window

Stop and Wait

Gönderdiğim her mesaj için durup ACK bekliyorum.

Maks 15ms'de gidip 15ms'de ACK dönmeliyse timeout = 30ms deriz.

30 ms'de ACK gelmezse, ya da NACK gelirse paketi tekrar yollarım.

NACK mesajın hatalı gittiği anlatır. Tek mesaj gönderdiğimden hatalı olan da o mesajdır.

Tekrar göndermem gereken de o mesajdır.

3 timeout nedeni olabilir:

- mesaj ulaşmamıştır.
- mesaj doğru ulaşmıştır. ACK bana ulaşmamıştır. → boşuna tekrar gönderdik aslında.
- mesaj yanlış ulaşmıştır. NACK bana ulaşmamıştır.

Avantajları:

- Paketler küçük parçalardan oluşur. Gönderip beklediğimiz için süre kısalsın diye.
- Maksimum 1 veri bufferda bekleyebileceği için, buffer alanı daha optimize'dir.
- İletişim ortamı (medium) daha kısa süreli mesguldür.
- Hata ihtimali azalır. (Paket küçük olduğundan)
- Hata kontrolü süresi kısalmış. Processing power. (Paket küçük olduğundan)
- Diğer cihazlara sıra daha cabuk gelir. Poll / SELECT yönteminde mesela.

Stop and Wait Line Utilization (U) Rate

Hattı ne kadar kullandığımızın hesabı.

t_{frame} : tek bir frame'in aktarılması için geçen süre.

gönderme / alma aşamasındaki işlem süreleri ve bufferda bekleme süresi buna dahil.

t_{prop} : göndericiden çıktıktan sonra alıcıya varması için geçen süre.

t_{ack} : alıcıdan gönderilecek olan ACK mesajının son bitinin çıkması için geçen süre.

ACK'nın ilk biti ile son biti arasındaki süre diyebiliriz.

→ Tüm aktarım için gerekli süre.

$$T_f = t_{\text{frame}} + t_{\text{prop}} + t_{\text{ack}} + t_{\text{prop}} \\ = t_{\text{frame}} + 2t_{\text{prop}}$$

→ çok küçük olduğundan genelde formülde yer almaz.

→ Line utilization

$$U = \frac{t_{\text{frame}}}{t_{\text{frame}} + 2t_{\text{prop}}} = \frac{\text{frame gönderme zamanı}}{\text{tüm geçen zaman}}$$

$$a = \frac{t_{\text{prop}}}{t_{\text{frame}}} \text{ dersek, } U = \frac{1}{1+2a}$$

ilk bit ulaşana kadar geçen süreye t_{prop} ,

sonrasında tüm frame aktarılınca

kadar geçen süreye t_{frame} ,

ACK'nın son biti karsıdan çıkana kadar

geçen süreye t_{ack} ,

ACK'nın ilk biti ulaşana kadar

geçen süreye t_{prop} diyerek;

toplam zamanı buluyoruz.

$$t_{\text{prop}} = \frac{\text{distance}}{\text{velocity}} = \frac{d}{v}$$

$$t_{\text{frame}} = \frac{\text{frameSize}}{\text{frameRate}}$$

ör 1000km arasında veri göndericez. ($d = 1000 \text{ km} = 10^6 \text{ m}$) distance

Hız 155.52 Mbps. ($R = 155.52 \cdot 10^6 \text{ bit/sec}$) dataRate

Hattın transmission hızı 200 000 000 m/sec ($v = 2 \cdot 10^8 \text{ m/sec}$) velocity

Her frame'in boyutu 424 bits. ($L = 424 \text{ bit}$) frameSize

Stop and Wait flow control modunda Line utilization nedir?

$$t_{\text{prop}} = \frac{\text{distance}}{\text{velocity}} = \frac{d}{v} = \frac{10^6 \text{ m}}{2 \cdot 10^8 \text{ m/sec}} = 1/200 \text{ sec}$$

$$t_{\text{frame}} = \frac{\text{frameSize}}{\text{frameRate}} = \frac{424 \text{ bit}}{155.52 \cdot 10^6 \text{ bit/sec}} = 2.73 \times 10^{-6} \text{ sec}$$

$$a = \frac{t_{\text{prop}}}{t_{\text{frame}}} \rightarrow U = \frac{1}{1+2a}$$

$$a = \frac{1/200 \text{ sec}}{2.73 \times 10^{-6} \text{ sec}} = 1834$$

$$U = \frac{1}{1+2 \cdot 1834} = 2.73 \times 10^{-4}$$

Sliding Window

Stop and wait yönteminde line utilization düşük olur. Verimli değil.

40l 100km olsun, aynı anda tek araba var gibi düşünebiliriz.

- Veriyi frame'lere bölüp, her biri için ACK almadan peşpeşe gönderiyorum. (max window boyutu kadar)
- Karşı taraf her frame başına ACK göndermek zorunda değil.

Bir ACK içinde birkaç frame için ACK bilgisi verebilir. Toplu ACK.

- Karşı tarafa giden bitler konvoy gibi gider.
- Frame'ler numaralandırılıp gönderilir. Farklı yollardan gidebilirler. **Numara gerekli.**

Aradan biri gelmemiş olabilir. Gönderildiği sırayla alınmamış olabilir.

n bit = 2^n frame ifade edebilir.

- Piggy Backing

Karşılıklı duplex bir iletişim var. ACK boyutu küçük.

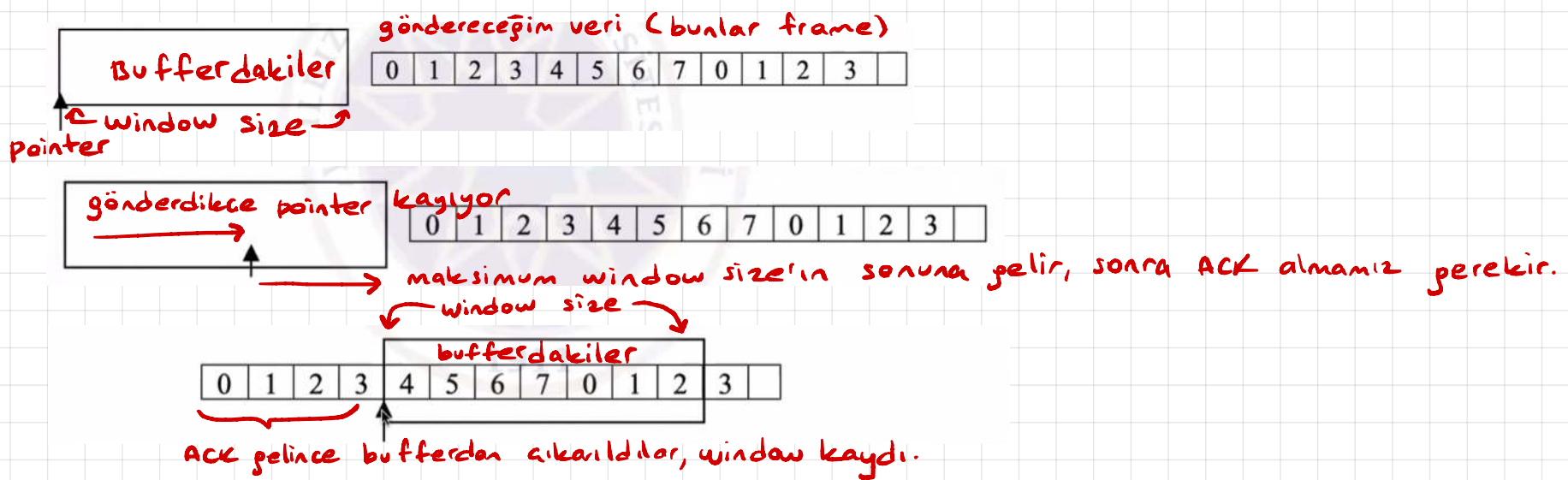
0 yüzden genelde karşı tarafa gönderdiğim ACK-NACK mesajlarını zaten karşı tarafa gönderdiğim veri mesajlarının içine ekliyorum.

Buna piggy backing deniyor.

Window numarası ve frame numarası birlikte tekil bir frame'i gösterir. ö: 3.windowun 2.frame'i.

Sender Side

- window size $2^n - 1$ or: $n=3$ iain, window size $2^3 - 1 = 7$



Receiver Side

